

Sistema de gestión de tienda online de electrónicos

Shomara Berenice Rivera Huerta
Enlace al repositorio en GitHub

12 de julio de 2026

Índice

1. Introducción y objetivos	3
2. Análisis y diseño	3
2.1. Diagrama entidad-relación (ER)	3
2.2. Justificación de la normalización	3
3. Implementación de la base de datos	5
4. Consultas y procedimientos almacenados	8
4.1. Consultas SQL	8
4.2. Procedimientos almacenados (Store Procedures)	9
4.3. Enlace al video demostrativo del proyecto	13
5. Resultados de pruebas y análisis de rendimiento	13
5.1. Resultados de consultas con datos de prueba	13
5.2. Análisis de rendimiento con índices (EXPLAIN)	15
5.3. Pruebas de restricciones y reglas de negocio	16
6. Conclusiones y lecciones aprendidas	18

Lista de códigos

1. Script de creación de esquema, restricciones e índices	5
2. Script de inserción de datos de prueba	6
3. Consulta 1: Listar productos disponibles por categoría, ordenados por precio.	8
4. Consulta 2: Mostrar clientes con pedidos pendientes y total de compras.	8
5. Consulta 3: Reporte de los 5 productos con mejor calificación promedio en reseñas.	8

6.	Store Procedures para automatizar reglas de negocio y mantener integridad de datos	9
7.	Prueba de reseña inválida	16
8.	Prueba de stock insuficiente	16
9.	Prueba de control de duplicados	16
10.	Prueba de resta de inventario	16
11.	Prueba de actualización de estatus	17
12.	Prueba de eliminación de opinión	17
13.	Prueba de modificación de perfil	17
14.	Prueba de alerta de inventario bajo	17

1. Introducción y objetivos

Actualmente las tiendas de productos electrónicos necesitan sistemas eficientes para gestionar su gran volumen de operaciones diarias, llevar el control manual del inventario, los clientes y las ventas, ésto suele generar errores como venta de productos sin stock o historiales de compras perdidos, por lo que en el desarrollo del siguiente proyecto, buscaremos optimizar la administración de la información mediante una base de datos relacional y normalizada, que permita a los administradores y clientes interactuar con el sistema de manera rápida y segura.

El objetivo de este sistema es centralizar la información mediante entidades bien definidas y el uso de llaves foráneas (*Foreign Keys*) que mantengan la integridad referencial, además, se busca automatizar las reglas de negocio, como el límite de pedidos pendientes y la validación de compras para las reseñas, utilizando procedimientos almacenados (*Store Procedures*) e índices para asegurar un rendimiento óptimo.

2. Análisis y diseño

Para evitar la redundancia de datos y proteger la integridad de la base de datos, se estructuró el modelo dividiendo la información en entidades conectadas lógicamente.

2.1. Diagrama entidad-relación (ER)

Se determinó una estructura compuesta por 6 entidades principales:

- **Categorías:** Organiza los productos.
- **Productos:** Almacena el stock y los precios.
- **Clientes:** Registra la información de contacto de los compradores.
- **Pedidos:** Gestiona el estado y fecha de las compras.
- **Detalles_pedido:** Entidad puente que rompe la relación de muchos a muchos entre pedidos y productos, detallando cantidades.
- **Resenas:** Guarda las opiniones de los usuarios sobre los productos.

2.2. Justificación de la normalización

Al analizar el diagrama Entidad-Relación diseñado, logré identificar que cumple con las formas normales basándonos en las reglas:

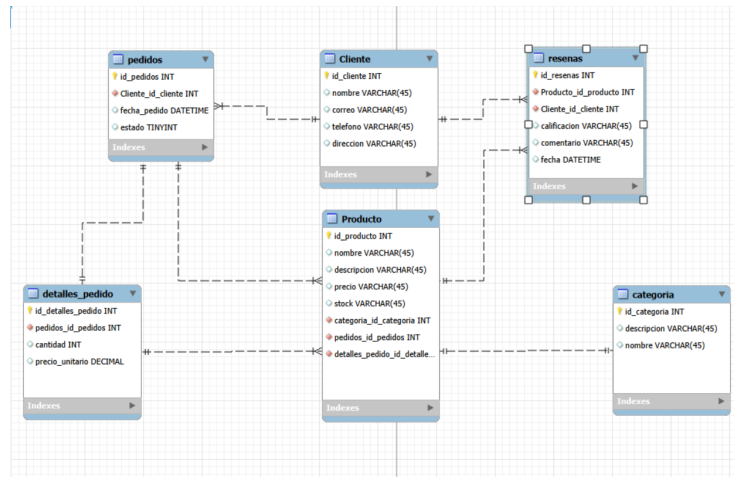


Figura 1: Diagrama entidad-relación del Sistema de tienda online

1. **Primera Forma Normal (1NF):** El diagrama muestra que todos los atributos de cada entidad son atómicos, o sea que no tiene atributos multivaluados, por ejemplo, en lugar de guardar una lista de productos dentro de un mismo pedido, se creó la tabla `detalles_pedido`. Cada entidad está diseñada para almacenar un único valor por campo, cumpliendo la 1NF.
2. **Segunda Forma Normal (2NF):** El modelo cumple con la 2NF porque además de cumplir el requisito de satisfacer la 1FN, no tiene dependencias parciales; en las entidades que funcionan como puente (como `detalles_pedido`), los atributos dependen de la llave completa que identifica a ese detalle, y no solo de una parte de ella.
3. **Tercera Forma Normal (3NF):** Cumple con la 3NF ya que no existen dependencias transitivas; al observar las entidades, nos damos cuenta de que todos los atributos dependen solo de su llave principal, ningún atributo no clave depende de otro atributo no clave, en la tabla `productos` no ponemos el nombre de la categoría, solo su llave foránea.
4. **Cuarta Forma Normal (4NF):** Cumple con la 4NF porque no tenemos ninguna tabla que intente almacenar listas de datos independientes al mismo tiempo, quiere decir que no mezclamos cosas que no tienen nada que ver entre sí en un mismo lugar, separamos los pedidos de las reseñas para evitar que se generen combinaciones de datos sin sentido.
5. **Quinta Forma Normal (5NF):** Cumple con la 5NF porque las relaciones entre nuestras entidades son directas, paso a paso y de dos en dos (por ejemplo: el Cliente se relaciona con su Pedido, y el Pedido con el Producto a través del detalle), no tenemos relaciones donde 3 entidades distintas dependan estrictamente unas de otras al mismo tiempo.

3. Implementación de la base de datos

A continuación, se presentan los scripts para la creación de las tablas, la optimización con índices y el código de la inserción de datos de prueba.

Código 1: Script de creación de esquema, restricciones e índices

```
1
2 CREATE DATABASE IF NOT EXISTS tienda_electronica;
3 USE tienda_electronica;
4
5 -- Categorías
6 CREATE TABLE categorias (
7     id_categoria INT AUTO_INCREMENT PRIMARY KEY,
8     nombre VARCHAR(100) NOT NULL UNIQUE, -- Nombre único
9     descripcion TEXT
10 );
11
12 -- Clientes
13 CREATE TABLE clientes (
14     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
15     nombre VARCHAR(150) NOT NULL,
16     correo VARCHAR(150) NOT NULL UNIQUE, -- Correo único
17     telefono VARCHAR(20),
18     direccion VARCHAR(255)
19 );
20
21 -- Productos
22 CREATE TABLE productos (
23     id_producto INT AUTO_INCREMENT PRIMARY KEY,
24     nombre VARCHAR(150) NOT NULL,
25     descripcion TEXT,
26     precio DECIMAL(10,2) NOT NULL,
27     stock INT NOT NULL CHECK (stock >= 0), -- Stock no negativo
28     id_categoria INT,
29     FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria)
30 );
31
32 -- Pedidos
33 CREATE TABLE pedidos (
34     id_pedido INT AUTO_INCREMENT PRIMARY KEY,
35     fecha_pedido DATE NOT NULL,
36     estado ENUM('pendiente', 'enviado', 'entregado') DEFAULT 'pendiente',
37     id_cliente INT,
38     FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
39 );
40
41 -- Detalles_pedido
42 CREATE TABLE detalles_pedido (
43     id_detalle INT AUTO_INCREMENT PRIMARY KEY,
44     id_pedido INT,
45     id_producto INT,
46     cantidad INT NOT NULL CHECK (cantidad > 0),
47     precio_unitario DECIMAL(10,2) NOT NULL,
48     FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido),
49     FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
```

```

50 );
51
52 -- Resenas
53 CREATE TABLE resenas (
54     id_resena INT AUTO_INCREMENT PRIMARY KEY,
55     calificacion INT NOT NULL CHECK (calificacion BETWEEN 1 AND 5),
56     comentario TEXT,
57     fecha DATE NOT NULL,
58     id_producto INT,
59     id_cliente INT,
60     FOREIGN KEY (id_producto) REFERENCES productos(id_producto),
61     FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)
62 );
63
64 -- Optimizar la búsqueda de productos por nombre
65 CREATE INDEX idx_producto_nombre ON productos(nombre);
66
67 -- Optimizar la búsqueda de productos por su categoría
68 CREATE INDEX idx_producto_categoria ON productos(id_categoria);
69
70 -- Optimizar la búsqueda del historial de pedidos de un cliente
    específico
71 CREATE INDEX idx_pedido_cliente ON pedidos(id_cliente);

```

Código 2: Script de inserción de datos de prueba

```

1  USE tienda_electronica;
2
3  -- Categorías (3)
4  INSERT INTO categorias (nombre, descripcion) VALUES
5  ('Teléfonos', 'Smartphones y accesorios celulares'),
6  ('Laptops', 'Computadoras portátiles para uso personal y
    profesional'),
7  ('Accesorios', 'Cables, audífonos, fundas y periféricos');
8
9  -- Clientes (15)
10 INSERT INTO clientes (nombre, correo, telefono, direccion) VALUES
11 ('Pablo Hernandez', 'pablo@email.com', '5551234', 'Calle 1'),
12 ('Jhovanny Pérez', 'jhovanny@email.com', '5551235', 'Calle 2'),
13 ('Zury Bautista', 'zury@email.com', '5551236', 'Calle 3'),
14 ('Juan Ruiz', 'juan@email.com', '5551237', 'Calle 4'),
15 ('Angel Duenas', 'angel@email.com', '5551238', 'Calle 5'),
16 ('Hugo Soto', 'hugo@email.com', '5551239', 'Calle 6'),
17 ('Sara Rivera', 'sara@email.com', '5551240', 'Calle 7'),
18 ('Paco Luna', 'paco@email.com', '5551241', 'Calle 8'),
19 ('Cecilia Hernández', 'cecilia@email.com', '5551242', 'Calle 9'),
20 ('Raúl Mora', 'raul@email.com', '5551243', 'Calle 10'),
21 ('Nolan Rivera', 'nolan@email.com', '5551244', 'Calle 11'),
22 ('Omar Navarrete', 'omar@email.com', '5551245', 'Calle 12'),
23 ('Lupita Gonzalez', 'lupita@email.com', '5551246', 'Calle 13'),
24 ('José Ruz', 'jose@email.com', '5551247', 'Calle 14'),
25 ('Rogelio Navarrete', 'rogelio@email.com', '5551248', 'Calle 15');
26
27 -- Productos (30)(10 teléfonos, 10 laptops, 10 accesorios)
28 INSERT INTO productos (nombre, descripcion, precio, stock,
    id_categoria) VALUES
29 ('iPhone 13', 'Teléfono Apple', 15000, 20, 1), ('Galaxy S21', 'Telé

```

```

fono Samsung', 12000, 15, 1),
30 ('Pixel 6', 'Teléfono Google', 10000, 10, 1), ('Xiaomi Mi 11', 'Tel
    éfono Xiaomi', 8000, 25, 1),
31 ('Moto G100', 'Teléfono Motorola', 7500, 30, 1), ('OnePlus 9', 'Tel
    éfono OnePlus', 11000, 5, 1),
32 ('iPhone 14', 'Teléfono Apple', 18000, 10, 1), ('Galaxy S22', 'Telé
    fono Samsung', 14000, 12, 1),
33 ('Poco X3', 'Teléfono Poco', 5000, 40, 1), ('Huawei P40', 'Teléfono
    Huawei', 9000, 8, 1),
34 ('MacBook Air', 'Laptop Apple', 20000, 10, 2), ('Dell XPS 13', '
    Laptop Dell', 25000, 5, 2),
35 ('ThinkPad X1', 'Laptop Lenovo', 22000, 7, 2), ('HP Envy', 'Laptop
    HP', 18000, 12, 2),
36 ('Asus ZenBook', 'Laptop Asus', 17000, 9, 2), ('Acer Swift', '
    Laptop Acer', 15000, 15, 2),
37 ('MacBook Pro', 'Laptop Apple', 30000, 3, 2), ('Razer Blade', '
    Laptop Razer', 35000, 2, 2),
38 ('MSI Stealth', 'Laptop MSI', 28000, 4, 2), ('Lenovo Legion', '
    Laptop Gaming Lenovo', 24000, 6, 2),
39 ('AirPods Pro', 'Audífonos Apple', 5000, 50, 3), ('Galaxy Buds', '
    Audífonos Samsung', 3000, 40, 3),
40 ('Sony WH-1000XM4', 'Audífonos Sony', 6000, 15, 3), ('Cable USB-C',
    'Cable 2 metros', 200, 100, 3),
41 ('Cargador Rápido', 'Cargador 20W', 400, 80, 3), ('Funda iPhone 13'
    , 'Funda silicona', 300, 60, 3),
42 ('Mica de Cristal', 'Protector de pantalla', 150, 120, 3), ('
    PowerBank 10000mAh', 'Batería externa', 800, 30, 3),
43 ('Mouse Inalámbrico', 'Mouse Bluetooth', 500, 45, 3), ('Teclado Mec
    ánico', 'Teclado USB', 1200, 20, 3);
44
45 -- Pedidos (20)
46 INSERT INTO pedidos (fecha_pedido, estado, id_cliente) VALUES
47 ('2023-10-01', 'entregado', 1), ('2023-10-02', 'entregado', 2), ('
    2023-10-03', 'entregado', 3),
48 ('2023-10-04', 'enviado', 4), ('2023-10-05', 'pendiente', 5), ('
    2023-10-06', 'entregado', 6),
49 ('2023-10-07', 'enviado', 7), ('2023-10-08', 'pendiente', 8), ('
    2023-10-09', 'entregado', 9),
50 ('2023-10-10', 'enviado', 10), ('2023-10-11', 'pendiente', 11), ('
    2023-10-12', 'entregado', 12),
51 ('2023-10-13', 'enviado', 13), ('2023-10-14', 'pendiente', 14), ('
    2023-10-15', 'entregado', 15),
52 ('2023-10-16', 'enviado', 1), ('2023-10-17', 'pendiente', 2), ('
    2023-10-18', 'entregado', 3),
53 ('2023-10-19', 'enviado', 4), ('2023-10-20', 'pendiente', 5);
54
55 -- Detalles de Pedido (25)
56 INSERT INTO detalles_pedido (id_pedido, id_producto, cantidad,
    precio_unitario) VALUES
57 (1, 1, 1, 15000), (1, 21, 1, 5000), (2, 11, 1, 20000), (3, 2, 1,
    12000), (3, 22, 1, 3000),
58 (4, 12, 1, 25000), (5, 3, 2, 10000), (6, 23, 1, 6000), (7, 13, 1,
    22000), (8, 4, 1, 8000),
59 (9, 24, 3, 200), (10, 14, 1, 18000), (11, 5, 1, 7500), (12, 25, 2,
    400), (13, 15, 1, 17000),
60 (14, 6, 1, 11000), (15, 26, 1, 300), (16, 16, 1, 15000), (17, 7, 1,
    18000), (18, 27, 2, 150),

```

```

61 (19, 17, 1, 30000), (20, 8, 1, 14000), (20, 28, 1, 800), (15, 29,
    1, 500), (1, 30, 1, 1200);
62
63 -- Reseñas (10)
64
65
66 INSERT INTO resenas (calificacion, comentario, fecha, id_producto,
    id_cliente) VALUES
67 (5, 'Excelente teléfono, muy rápido.', '2023-10-10', 1, 1),
68 (4, 'Buena laptop, pero la batería dura poco.', '2023-10-11', 11,
    2),
69 (5, 'Me encantó la cámara del S21.', '2023-10-12', 2, 3),
70 (3, 'Los audífonos están bien, un poco caros.', '2023-10-13', 21,
    1),
71 (5, 'Muy potente para programar.', '2023-10-14', 12, 4),
72 (4, 'Buen diseño de audífonos.', '2023-10-15', 22, 3),
73 (5, 'Carga súper rápido.', '2023-10-16', 25, 12),
74 (2, 'La funda se rayó muy rápido.', '2023-10-17', 26, 15),
75 (5, 'El mejor mouse que he tenido.', '2023-10-18', 29, 15),
76 (4, 'Buen teclado, aunque algo ruidoso.', '2023-10-19', 30, 1);

```

4. Consultas y procedimientos almacenados

Para manipular los datos y aplicar las reglas de negocio, se diseñaron consultas y procedimientos almacenados para no depender de la aplicación externa.

4.1. Consultas SQL

Se desarrollaron SELECT con JOIN y funciones de agregación (COUNT, AVG).

Código 3: Consulta 1: Listar productos disponibles por categoría, ordenados por precio.

```

1 SELECT p.nombre AS producto, c.nombre AS categoria, p.precio, p.
    stock
2 FROM productos p
3 JOIN categorias c ON p.id_categoria = c.id_categoria
4 WHERE p.stock > 0
5 ORDER BY p.precio ASC;

```

Código 4: Consulta 2: Mostrar clientes con pedidos pendientes y total de compras.

```

1 SELECT c.nombre AS cliente, c.correo, COUNT(pe.id_pedido) AS
    total_pedidos_pendientes
2 FROM clientes c
3 JOIN pedidos pe ON c.id_cliente = pe.id_cliente
4 WHERE pe.estado = 'pendiente'
5 GROUP BY c.id_cliente;

```

Código 5: Consulta 3: Reporte de los 5 productos con mejor calificación promedio en reseñas.


```

1 SELECT p.nombre AS producto, AVG(r.calificacion) AS
   promedio_calificacion
2 FROM productos p
3 JOIN resenas r ON p.id_producto = r.id_producto
4 GROUP BY p.id_producto
5 ORDER BY promedio_calificacion DESC
6 LIMIT 5;

```

4.2. Procedimientos almacenados (Store Procedures)

Los procedimientos nos permiten validar condiciones usando IF y frenar transacciones inválidas con SIGNAL.

Código 6: Store Procedures para automatizar reglas de negocio y mantener integridad de datos

```

1 -- Procedimientos almacenados
2
3 -- Cambiamos el delimitador a dos diagonales para que mysql
   entienda que todo el procedimiento es un solo bloque de código
   y no se detenga cuando encuentre los puntos y comas internos
4 DELIMITER //
5
6 -- 1. Registrar un nuevo pedido, verificando el límite de 5 pedidos
   pendientes y stock suficiente.
7 -- este procedimiento sirve para automatizar las compras porque
   primero declaramos variables locales para guardar de forma
   temporal los datos que vamos a consultar de las tablas, luego
   usamos SELECT COUNT y SELECT STOCK INTO para meter los
   resultados dentro de nuestras variables y poder hacer las
   validaciones del negocio con un IF, si el cliente ya debe 5
   pedidos o si no hay stock, usamos signal SQLSTATE '45000' para
   detener todo por completo y lanzar un error personalizado,
   evitando que la base de datos se rompa o registre datos falsos;
   si todo esta bien, se inserta el pedido y usamos
   LAST_INSERT_ID para agarrar el id que se acaba de generar en
   automatico y meterlo directo en detalles_pedido
8 CREATE PROCEDURE sp_registrar_pedido(
9     IN p_id_cliente INT,
10    IN p_id_producto INT,
11    IN p_cantidad INT,
12    IN pPrecio DECIMAL(10,2)
13 )
14 BEGIN
15     DECLARE v_pendientes INT;
16     DECLARE v_stock INT;
17     DECLARE v_nuevo_id_pedido INT;
18
19     -- contamos cuantos pedidos pendientes tiene este cliente y lo
   guardamos en la variable
20     SELECT COUNT(*) INTO v_pendientes FROM pedidos WHERE id_cliente
   = p_id_cliente AND estado = 'pendiente';
21
22     -- revisamos cuanto stock tiene el producto actual y lo
   guardamos en la variable

```

```

23 SELECT stock INTO v_stock FROM productos WHERE id_producto =
    p_id_producto;
24
25 IF v_pendientes >= 5 THEN
26     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: El
    cliente ya tiene 5 pedidos pendientes máximos.';
27 ELSEIF v_stock < p_cantidad THEN
28     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: No hay
    suficiente stock para este producto.';
29 ELSE
30     -- si pasa los filtros guardamos el pedido usando CURDATE
    para poner la fecha del día de hoy en automatico
31     INSERT INTO pedidos (fecha_pedido, estado, id_cliente)
    VALUES (CURDATE(), 'pendiente', p_id_cliente);
32
33     -- guardamos el id generado para no perder la relacion con
    su detalle
34     SET v_nuevo_id_pedido = LAST_INSERT_ID();
35
36     -- insertamos el producto en la tabla puente de
    detalles_pedido
37     INSERT INTO detalles_pedido (id_pedido, id_producto,
    cantidad, precio_unitario) VALUES (v_nuevo_id_pedido,
    p_id_producto, p_cantidad, p_precio);
38 END IF;
39 END //
40
41
42 -- 2. Registrar una reseña, verificando que el cliente haya
    comprado el producto.
43 -- este procedimiento protege la integridad de las opiniones de la
    tienda, hacemos un SELECT COUNT cruzando detalles_pedido y
    pedidos con un JOIN para verificar si existe al menos un
    registro donde este cliente haya pagado por este producto, si
    el conteo da cero significa que nunca lo compro, por lo que
    usamos SIGNAL SQLSTATE para arrojar un error y bloquear la
    insercion de la resena
44 CREATE PROCEDURE sp_registrar_resena(
45     IN p_calificacion INT,
46     IN p_comentario TEXT,
47     IN p_id_producto INT,
48     IN p_id_cliente INT
49 )
50 BEGIN
51     DECLARE v_compro INT;
52
53     -- buscamos si hay registros que demuestren que el cliente si
    compro el producto
54     SELECT COUNT(*) INTO v_compro
55     FROM detalles_pedido dp
56     JOIN pedidos p ON dp.id_pedido = p.id_pedido
57     WHERE p.id_cliente = p_id_cliente AND dp.id_producto =
    p_id_producto;
58
59     IF v_compro = 0 THEN
60         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Solo
    puedes hacer reseñas de productos que hayas comprado.';

```

```

61 ELSE
62     INSERT INTO resenas (calificacion, comentario, fecha,
63         id_producto, id_cliente)
64     VALUES (p_calificacion, p_comentario, CURDATE(),
65         p_id_producto, p_id_cliente);
66 END IF;
67 END //
68
69 -- 3. Actualizar el stock de un producto después de un pedido
70 -- este procedimiento sirve para manteer actualizado el inventario,
71 -- lo que hacemos es un UPDATE directo a la tabla de productos
72 -- usando su llave primaria y le restamos al stock original la
73 -- cantidad exacta de piezas que el cliente se va a llevar en su
74 -- compra
75 CREATE PROCEDURE sp_actualizar_stock(
76     IN p_id_producto INT,
77     IN p_cantidad_comprada INT
78 )
79 BEGIN
80     UPDATE productos
81     SET stock = stock - p_cantidad_comprada
82     WHERE id_producto = p_id_producto;
83 END //
84
85 -- 4. Cambiar el estado de un pedido (ej. de pendiente a enviado)
86 -- sirve para la logica de logistica de la tienda, usamos un update
87 -- para modificar el texto del estado de la orden (pendiente,
88 -- enviado, entregado) guiandonos por el id_pedido para no
89 -- alterar las compras de otros clientes
90 CREATE PROCEDURE sp_cambiar_estado_pedido(
91     IN p_id_pedido INT,
92     IN p_nuevo_estado VARCHAR(20)
93 )
94 BEGIN
95     UPDATE pedidos
96     SET estado = p_nuevo_estado
97     WHERE id_pedido = p_id_pedido;
98 END //
99
100 -- 5. Eliminar reseña de un producto específico, actualizando el
101 -- promedio de calificaciones
102 -- aqui primero buscamos cual es el id del producto al que
103 -- pertenece la reseña que queremos borrar y lo respaldamos en una
104 -- variable local, ya que si la borramos primero perderiamos ese
105 -- dato; despues de aplicar el delete, hacemos un select combinado
106 -- con AVG para mostrarle de inmediato al administrador como se
107 -- recalculo el promedio general de estrellas de ese producto de
108 -- manera automatica
109 CREATE PROCEDURE sp_eliminar_resena(
110     IN p_id_resena INT
111 )
112 BEGIN
113     DECLARE v_id_producto INT;

```

```

102 -- respaldamos el id del producto en nuestra variable antes de
    borrar el registro
103 SELECT id_producto INTO v_id_producto FROM resenas WHERE
    id_resena = p_id_resena;
104
105 -- borramos la resena de la tabla
106 DELETE FROM resenas WHERE id_resena = p_id_resena;
107
108 -- calculamos y mostramos el nuevo promedio real en pantalla
109 SELECT p.nombre, AVG(r.calificacion) AS nuevo_promedio
110 FROM productos p
111 LEFT JOIN resenas r ON p.id_producto = r.id_producto
112 WHERE p.id_producto = v_id_producto
113 GROUP BY p.id_producto;
114 END //
115
116
117 -- 6. Agregar un nuevo producto, verificando que no exista un
    duplicado (mismo nombre y categoría).
118 -- para evitar errores de captura donde metan dos veces la misma
    laptop o celular, hacemos un select count buscando si ya hay un
    registro con exactamente el mismo nombre dentro de la misma
    categoría, si el sistema encuentra que ya existe, frena la
    operación con un signal y le avisa al usuario
119 CREATE PROCEDURE sp_agregar_producto(
120     IN p_nombre VARCHAR(150),
121     IN p_descripcion TEXT,
122     IN p_precio DECIMAL(10,2),
123     IN p_stock INT,
124     IN p_id_categoria INT
125 )
126 BEGIN
127     DECLARE v_existe INT;
128
129     SELECT COUNT(*) INTO v_existe FROM productos
130     WHERE nombre = p_nombre AND id_categoria = p_id_categoria;
131
132     IF v_existe > 0 THEN
133         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Ya
            existe un producto con ese nombre en esta categoría.';
134     ELSE
135         INSERT INTO productos (nombre, descripcion, precio, stock,
            id_categoria)
136         VALUES (p_nombre, p_descripcion, p_precio, p_stock,
            p_id_categoria);
137     END IF;
138 END //
139
140
141 -- 7. Actualizar la información de un cliente (ej. dirección o telé
    fono)
142 -- este procedimiento sirve para que el usuario pueda cambiar sus
    datos de contacto en su perfil, aplicamos un UPDATE directo
    sobre la tabla clientes y usamos el id_cliente en el WHERE para
    que se modifiquen los datos de la persona correcta
143 CREATE PROCEDURE sp_actualizar_cliente(
144     IN p_id_cliente INT,

```

```

145     IN p_nueva_direccion VARCHAR(255),
146     IN p_nuevo_telefono VARCHAR(20)
147 )
148 BEGIN
149     UPDATE clientes
150     SET direccion = p_nueva_direccion, telefono = p_nuevo_telefono
151     WHERE id_cliente = p_id_cliente;
152 END //
153
154
155 -- 8. Generar un reporte de productos con stock bajo (menos de 5
156 -- unidades)
157 -- es una consulta de control para el administrador de la tienda,
158 -- hace un SELECT filtrando con un WHERE para extraer unicamente
159 -- los productos que tengan menos de 5 piezas en el inventario,
160 -- ordenandolos de menor a mayor stock para saber que urge
161 -- resurtir primero.
162 CREATE PROCEDURE sp_reporte_stock_bajo()
163 BEGIN
164     SELECT nombre, stock, precio
165     FROM productos
166     WHERE stock < 5
167     ORDER BY stock ASC;
168 END //
169
170 -- regresamos el delimitador al punto y coma que se usa de forma
171 -- normal en sql
172 DELIMITER ;

```

4.3. Enlace al video demostrativo del proyecto

Para comprobar de forma visual y práctica el correcto funcionamiento de todo el sistema relacional, los scripts y las búsquedas en vivo dentro de MySQL Workbench, se grabó una demostración en video. El archivo multimedia se encuentra en Google Drive y puede ser consultado a través del siguiente enlace directo:

LINK_DE_VIDEO_DE_DRIVE

5. Resultados de pruebas y análisis de rendimiento

La base de datos fue sometida a una validación ejecutando consultas y simulando escenarios de error mediante el comando `CALL`, para comprobar que el sistema realmente soporta el uso diario de una tienda online.

5.1. Resultados de consultas con datos de prueba

Al realizar las consultas que construimos en el Punto 4 Fase 3 de este proyecto con los datos de prueba ingresados, el sistema nos devolvió exactamente la información que esperábamos:

1. **Productos disponibles por categoría:** Al usar la cláusula JOIN, el sistema cruzó las tablas correctamente para mostrarnos el nombre de la categoría en lugar de solo su ID numérico, ordenando los precios de menor a mayor.

producto	categoria	precio	stock
Mica de Cristal	Accesorios	150.00	120
Cable USB-C	Accesorios	200.00	100
Funda iPhone 13	Accesorios	300.00	60
Cargador Rápido	Accesorios	400.00	80
Mouse Inalámbrico	Accesorios	500.00	45
PowerBank 10000mAh	Accesorios	800.00	30
Teclado Mecánico	Accesorios	1200.00	20
Galaxy Buds	Accesorios	3000.00	40
AirPods Pro	Accesorios	5000.00	50
Poco X3	Teléfonos	5000.00	40
Sony WH-1000XM4	Accesorios	6000.00	15
Moto G100	Teléfonos	7500.00	30
Xiaomi Mi 11	Teléfonos	8000.00	25
Huawei P40	Teléfonos	9000.00	8
Pixel 6	Teléfonos	10000.00	10
OnePlus 9	Teléfonos	11000.00	5
Galaxy S21	Teléfonos	12000.00	15
Galaxy S22	Teléfonos	14000.00	12
Acer Swift	Laptops	15000.00	15
iPhone 13	Teléfonos	15000.00	20
Asus ZenBook	Laptops	17000.00	9
HP Envy	Laptops	18000.00	12
iPhone 14	Teléfonos	18000.00	10
MacBook Air	Laptops	20000.00	10
ThinkPad X1	Laptops	22000.00	7
Lenovo Legion	Laptops	24000.00	6
Dell XPS 13	Laptops	25000.00	5
MSI Stealth	Laptops	28000.00	4
MacBook Pro	Laptops	30000.00	3
Razer Blade	Laptops	35000.00	2

2. **Clientes con pedidos pendientes:** El sistema agrupó correctamente los datos de los nuevos clientes usando GROUP BY y contó cuántas compras en proceso tiene cada uno de forma individual.

cliente	correo	total_pedidos_pendientes
---------	--------	--------------------------

Angel Duenas	angel@email.com		2
Paco Luna	paco@email.com		1
Nolan Rivera	nolan@email.com		1
José Ruz	jose@email.com		1
Jhovanny Pérez	jhovanny@email.com		1

3. **Mejores calificaciones promedio:** La función AVG calculó los promedios reales y el LIMIT 5 funcionó a la perfección para traernos solo el top de productos.

producto	promedio_calificacion
iPhone 13	5.0000
Mouse Inalámbrico	5.0000
Galaxy S21	5.0000
Cargador Rápido	5.0000
Dell XPS 13	5.0000

5.2. Análisis de rendimiento con índices (EXPLAIN)

Como vimos en las diapositivas de clase, los índices funcionan exactamente como el índice de un libro, permitiendo que MySQL vaya directo a la fila que buscamos en lugar de leer toda la tabla desde el principio; para comprobar que funcionan, usamos el comando EXPLAIN antes de hacer un SELECT, lo cual nos sirve para ver el comportamiento del sistema a detalle.

```
mysql> EXPLAIN SELECT * FROM productos WHERE nombre = 'iPhone 13';
```

id	select_type	table	type	possible_keys	key	key_len
1	SIMPLE	productos	ref	idx_producto_nombre	idx_producto_nombre	602

La explicación de esta tabla es la siguiente:

- En la columna **key**, notamos que el sistema sí utilizó nuestro índice llamado **idx_producto_nombre**.
- En la columna **rows**, vemos el número 1. Esto significa que MySQL solo tuvo que escanear una única fila para encontrar el iPhone 13, demostrando que la consulta es súper rápida y no desperdicia recursos del servidor.

5.3. Pruebas de restricciones y reglas de negocio

Realizamos llamadas a los procedimientos almacenados intentando ingresar datos falsos o inválidos intencionalmente, para comprobar si nuestras alertas lograban detenerlos.

- **Intento de reseña sin compra (sp_registrar_resena):** Intentamos que el cliente 2 (Jhovanny Pérez) hiciera una reseña de un producto que no ha pagado.

Código 7: Prueba de reseña inválida

```
1 CALL sp_registrar_resena(5, 'Excelente celular', 1, 2);
```

Resultado: El sistema detectó que las tablas no cruzaban a través del JOIN y bloqueó la inserción arrojando el mensaje: *Error: Solo puedes hacer reseñas de productos que hayas comprado.*

- **Intento de compra sin stock (sp_registrar_pedido):** Tratamos de insertar un pedido pidiendo 1000 unidades de un artículo que solo tenía 20.

Código 8: Prueba de stock insuficiente

```
1 CALL sp_registrar_pedido(1, 1, 1000, 15000.00);
```

Resultado: El procedimiento ejecutó nuestro condicional IF deteniendo la venta y respondió: *Error: No hay suficiente stock para este producto.*

- **Intento de agregar producto duplicado (sp_agregar_producto):** Quisimos registrar un artículo con el mismo nombre y categoría que uno ya existente.

Código 9: Prueba de control de duplicados

```
1 CALL sp_agregar_producto('iPhone 13', '...', 15000, 10, 1);
```

Resultado: El conteo interno dio mayor a cero, por lo que se activó el SIGNAL SQLSTATE mostrando el error: *Error: Ya existe un producto con ese nombre en esta categoría.*

- **Actualización automática de inventario (sp_actualizar_stock):** Probamos la función restándole 5 piezas al stock del producto 2 tras una compra simulada.

Código 10: Prueba de resta de inventario

```
1 CALL sp_actualizar_stock(2, 5);
```

Resultado: El sistema aplicó la operación matemática de resta de forma correcta en la fila correspondiente de la tabla productos.

- **Cambio en el estatus logístico (sp_cambiar_estado_pedido):** Invocamos el procedimiento para pasar la orden de compra número 1 al estatus de enviada.

Código 11: Prueba de actualización de estatus

```
1 CALL sp_cambiar_estado_pedido(1, 'enviado');
2 \end'
```

Resultado: El UPDATE se realizó con éxito y la orden quedó marcada para su distribución en el sistema.

- **Borrado de reseña y recálculo (sp_eliminar_resena):** Eliminamos la opinión con ID 1 para auditar que el promedio de estrellas cambiara al instante.

Código 12: Prueba de eliminación de opinión

```
1 CALL sp_eliminar_resena(1);
```

Resultado: MySQL borró el registro y ejecutó la función AVG en pantalla, entregándole al administrador el promedio real actualizado.

- **Modificación de perfil de cliente (sp_actualizar_cliente):** Cambiamos la dirección del cliente 1 a 'Avenida Principal 123' por una mudanza.

Código 13: Prueba de modificación de perfil

```
1 CALL sp_actualizar_cliente(1, 'Avenida Principal 123', '
555-4321');
```

Resultado: La información se actualizó en la tabla clientes respetando el ID de la persona sin alterar sus demás datos.

- **Extracción de reporte crítico (sp_reporte_stock_bajo):** Solicitamos la lista de artículos con inventario menor a 5 unidades.

Código 14: Prueba de alerta de inventario bajo

```
1 CALL sp_reporte_stock_bajo();
```

Resultado: El sistema filtró las filas y nos desplegó ordenados de menor a mayor los productos que están a punto de agotarse.

- **Restricción de integridad Check:** Intentamos hacer un INSERT forzado metiendo una reseña con calificación directa de 7 estrellas. **Resultado:** MySQL detuvo la inserción y marcó error de integridad debido a la restricción CHECK (calificacion BETWEEN 1 AND 5) definida en la tabla, protegiendo así el formato de los datos de forma automática.

6. Conclusiones y lecciones aprendidas

Hacer la implementación de esta base de datos paso a paso me permitió entender mucho mejor cómo se conecta toda la información del backend de una tienda real. Al analizar las tablas y programar los procedimientos, me di cuenta de que las llaves foráneas y la normalización son la pieza principal para que todo funcione en orden, ya que evitan que tengamos clientes fantasma o que modifiquemos el dato de una categoría y se desajuste en otros lados.

Otro punto importante que comprendí mejor con este proyecto fue que utilizar Procedimientos Almacenados en lugar de solo insertar datos a la fuerza, nos da una capa de seguridad gigante, pues gracias al uso de las condicionales y el comando **SIGNAL**, es prácticamente imposible meter "datos basura." romper las reglas del negocio, incluso si un empleado se equivoca al capturar.

Al ser mi primer proyecto completo de bases de datos, me llevo las siguientes lecciones aprendidas como principiante:

- **El diseño en papel ahorra horas de código:** Aprendí que tomarse el tiempo de hacer bien el diagrama Entidad-Relación y normalizar las tablas antes de abrir MySQL evita tener que borrar y rehacer toda la base de datos a la mitad del proyecto.
- **La automatización de los IDs:** Entendí por qué es mejor usar **INT AUTO_INCREMENT** para las llaves primarias; dejar que el sistema asigne los números por sí solo evita choques de información y hace los scripts de inserción mucho más limpios, esto lo noté porque al inicio del proyecto quería que todo fuera ordenado y visual para mi poniendo los ids combinados con letras para identificarlos pero sobre la marcha me di cuenta de que sería complicarme las cosas y me quedé con los **INT**.
- **Una base de datos no solo guarda información, la protege:** Descubrí que herramientas sencillas como el **CHECK** o hacer **JOIN** en los **STORED PROCEDURES** son como candados de seguridad, la base de datos es responsable de rechazar los datos ilógicos.
- **Aprender a leer los errores:** Al principio los errores en rojo de MySQL me desconcertaban porque no los entendía de todo, pero aprendí que leerlos detalladamente te dice exactamente dónde te equivocaste, especialmente cuando se trata de llaves foráneas que no coinciden.

En el futuro, como mejora adicional, propondría la creación de **VIEWS** para guardar las consultas que utilizan muchos **JOIN** (como los reportes de ventas), de esta forma sería mucho más sencillo y rápido para cualquier usuario leer la información sin tener que comprender código SQL complejo.

En conclusión, el sistema desarrollado funciona correctamente, cumple con los objetivos planteados y proporciona una base sólida para la gestión de una tienda online de electrónicos.